

VERIQO Constitutional Sequencing & Authority Audit

Round: CS (Constitutional Sequencing) **Date:** 2026-09-03 **Branch:** `claude/l99-gap-coverage-nv70zy` **Trigger:** a review conducted against the actual integration test and the runtime artefact, not against the report.

0. The finding that started this round, and why it lands

The reviewer did not read the closure report and grade it. They opened `evidence/RUNTIME_EVIDENCE.json` and `test/integration/system_integration_proof_test.go` and read the order of events. That is the only way the defect was findable, because every prose description of the architecture in the previous round was correct. The **code** was in the wrong order; the **words** were not.

Two things were found:

1. **`case.resolved` was emitted before `proof.sealed`, and the whole reverse run happened after `Case.Resolve()`.** Reverse proof was therefore a retrospective audit, not a constitutional gate.
2. **`caseproofgraph.Build()` called `proof.NewFinding`.** The graph, which the previous report described as "reads; decides nothing", was a second place a finding could come into existence.

Both were confirmed in the source before a single line was changed. Neither was a reporting error. They were real, and this document records what was done about them.

1. Defect one: reverse proof was post-hoc

1.1 What the artefact actually said

The **superseded** runtime artefact emitted the stream below. It is reproduced here only to record what the defect looked like; it is **not** what `evidence/RUNTIME_EVIDENCE.json` contains today. The current stream is in §1.5, and `TestNoDocumentMisquotesTheRuntimeLedger` fails if any document in this repository quotes a superseded ledger without saying so.

<!-- RUNTIME-LEDGER: historical -->

AUDIT-007	<code>qualification_begun</code>	<- SUPERSEDED. Not the current artefact.
AUDIT-008	<code>proof_attached</code>	<- SUPERSEDED.
AUDIT-009	<code>case.resolved</code>	<- SUPERSEDED. Resolution is now 013.
AUDIT-010	<code>proof.sealed</code>	<- SUPERSEDED.

A case resolved, and then its proof was sealed. The Case Fabric phase contract states the exit condition of `UNDER_QUALIFICATION` as *every material claim carries a sealed proof object*, and the entry condition of `RESOLVED` as *every material claim is proven*. The runtime stream violated both, and nothing in the system noticed, because nothing in the system was looking at order.

1.2 The root cause was not the test — it was `fref.Close`

The obvious repair is to move the reverse block higher in the integration test. That would have been a cosmetic fix that left the defect intact, because the test was written the way it was for a reason: `fref.Close` required a **complete forward run**:

```
// before
if err := fwd.RequireComplete(); err != nil { ... }
```

A reverse closure could not be computed until the forward run had already reached its final stage — which is to say, until the decision already existed. The precondition *forced* reverse proof to be post-hoc. Any test written against the old API would have had the same shape.

The precondition is now:

```
// after -- pkg/fref/fref.go
if !fwd.Reached(StageTrust) {
    return Closure{}, fmt.Errorf(...)
}
```

Reverse closure needs the evidence and the trust assessment. It does not need the decision it is supposed to gate. Fixing the premise made the lawful ordering expressible; the test was then rewritten to use it, not the other way round.

1.3 The law, in code

`pkg/fref/sequence.go` (new, 365 lines) declares the twelve-step constitutional sequence and, for each step, the steps that gate it:

1. CASE	(no gates: this is where a case begins)
2. SCOPE	gated on: CASE
3. EVIDENCE	gated on: SCOPE
4. HYPOTHESIS	gated on: EVIDENCE
5. REVERSE_PROOF	gated on: HYPOTHESIS
6. QUALIFICATION	gated on: REVERSE_PROOF
7. PROOF_SEAL	gated on: QUALIFICATION
8. FINDING	gated on: PROOF_SEAL
9. AUTHORIZED_DECISION	gated on: FINDING
10. CASE_RESOLUTION	gated on: REVERSE_PROOF, PROOF_SEAL, FINDING, AUTHORIZED_DECISION
11. LEDGER	gated on: CASE_RESOLUTION
12. REPLAY	gated on: LEDGER

`fref.ExplainSequence()` prints this, and ends with the sentence the law exists to enforce:

```
REVERSE_PROOF, PROOF_SEAL, FINDING and AUTHORIZED_DECISION all gate
CASE_RESOLUTION. Reverse proof is a constitutional gate, not a retrospective audit.
```

1.4 The hole the reviewer did not name, and why it mattered

`VerifyEventOrder([]string)` checks that the emitted ledger is in a lawful order. That is necessary and insufficient. **A stream that omits a gate entirely is perfectly ordered.** A ledger that runs

```
case.opened → case.scoped → case.evidence_pinned → case.hypothesis_recorded
→ case.resolved
```

has no ordering violation at all. It has skipped reverse proof, the seal, the finding and the authorized decision — every gate — and order alone cannot see it.

`VerifyEventGates([]string) []string` was added for exactly this. It reports each step present in the stream whose gates are **absent**. Both are asserted, per domain, in the integration test:

```
if v := fref.VerifyEventOrder(actions); len(v) > 0 {
    t.Fatalf("domain %q emitted an out-of-order ledger: %s", dc.domain, v[0])
}
if g := fref.VerifyEventGates(actions); len(g) > 0 {
```

```
t.Fatalf("domain %q skipped a constitutional gate: %s", dc.domain, g[0])
}
```

`TestAnUngatedLedgerIsCaughtEvenWhenPerfectlyOrdered` in the adversarial suite is the test that fails if `VerifyEventGates` is removed.

1.5 The runtime artefact now

`cmd/veriqo-runtime-evidence` runs both FREF directions before qualification begins, and verifies **order and gates** before it will emit anything. Thirteen records:

```
001 case.opened           008 case.qualification_began
002 case.scoped           009 case.proof_attached
003 case.evidence_pinned  010 proof.sealed
004 case.hypothesis_recorded 011 claim.finding_founded
005 case.claim_registered  012 case.decision_authorized
006 case.hypothesis_tested 013 case.resolved
007 qualification.reverse_closed
```

Reverse closure is record 007, before qualification at 008. Resolution is record 013, last. The defect the reviewer found is now the thing the generator refuses to produce.

A note on vocabulary: the first attempt at this introduced three new event families — `reverse.closed`, `finding.founded`, `decision.authorized` — and the closed 25-family taxonomy rejected all three. That refusal was correct. The events were mapped into the families that already existed (`qualification.`, `claim.`, `case.`) rather than the taxonomy being widened to accommodate a convenience.

1.6 `Seal()` already required reverse closure

The mandate (§10) asked that `Seal()` not merely look at the reverse-proof set statically but require `ReverseProof.Close == TRUE` for material claims. This was already the case and remains so — `deriveSufficiency` in `pkg/proof/proof.go` returns `Insufficient` when `ReverseProofGap.Complete` is false, and defers to the EQF's own judgement rather than recomputing it:

```
// The reverse proof must actually be complete. We defer to the EQF's
// own judgement here rather than re-deriving it ... a second opinion
// computed in this package would be a duplicate authority on the same
// question.
if !o.ReverseProofGap.Complete {
    return Insufficient
}
```

No change was needed. It is recorded here because the mandate asked for it to be confirmed, and confirming it is not the same as claiming to have built it.

2. Defect two: the graph was a second finding authority

2.1 The explicit answer to the explicit question

The reviewer asked, precisely:

```
Apakah caseproofgraph sedang "creating a finding", atau hanya "materializing an already-authorized finding representation"?
```

It was creating one. Not materializing. `addProof` in `pkg/caseproofgraph/build.go` contained a call to `proof.NewFinding(o, 0)`, guarded on `Sufficiency == Sufficient && Stance == Support`. That is a finding coming into existence inside a package whose stated role is composition and projection. The previous report's sentence "Build reads; it decides nothing" was false about this code path, and the reviewer was right to disbelieve it from the source rather than accept it from the prose.

The suspicion that it "might be a deterministic projection of an already-sealed proof" was the charitable reading, and it does not rescue the design: a deterministic function that constructs the authoritative object is still the second place that object can be born. Two constructors that agree today are two constructors.

2.2 What it does now

`Build` takes findings as an argument and materializes only:

```
func Build(c *casefabric.Case, proofs map[string]proof.Object,
          findings map[string]proof.Finding, tick uint64) (*Graph, error)
```

and in `addProof`:

```
if hasFinding && !f.IsZero() {
    if f.ProofHash() != o.CanonicalHash {
        return fmt.Errorf("caseproofgraph: the supplied finding for claim %q "+
            "belongs to proof %s, not %s", ...)
    }
    ...
}
```

`proof.NewFinding` is no longer **called** anywhere in `pkg/caseproofgraph` — the name survives only in two comments explaining why this package must not call it. The graph cannot produce a finding node without being handed a finding, and it refuses a finding that belongs to a different proof object.

The test the reviewer asked for exists under the name they proposed: `TestCaseProofGraphCannotCreateIndependentFindingAuthority`. It fails if a finding node appears in a graph built without one being supplied. `TestGraphAsABackDoorToAFinding` in the adversarial suite attacks the same boundary from the outside.

3. The ten laws, answered one at a time

The mandate set ten laws to verify and, where necessary, repair. Each is answered below with what enforces it and what fails if the enforcement is removed.

Law 1 — Reverse proof MUST close before resolution

Was violated. Now enforced.

`casefabric.Case.Resolve` takes a `ResolutionGate`:

```
type ResolutionGate struct {
    Decision          proof.Decision
    ReverseClosureHolds bool
    ClosureSubject    string
    ClosureExplanation string
}
```

`Resolve` refuses when `ReverseClosureHolds` is false, when the closure subject does not match the claim, or when the explanation is empty. The gate carries *evidence that the step happened*, not an assertion that it did.

Fails if removed: `TestResolveFirstProveLater`, `TestReverseAsAnAfterthought`, `TestReverseProofGatesResolution`.

Law 2 — Proof MUST be sealed before RESOLVED

Enforced. `Resolve` runs `requireAttachedProof` over every material claim; a claim whose attached object is unsealed (no canonical hash) blocks resolution. The check order inside `Resolve` is epistemic first — unproven or untested claims are reported before gate failures — so the error a caller sees names the real deficiency rather than the first structural tripwire.

At the ledger level, `PROOF_SEAL` gates `FINDING`, which gates `AUTHORIZED_DECISION`, which gates `CASE_RESOLUTION`. A stream with `case.resolved` before `proof.sealed` is rejected twice: once by order, once by gates.

Law 3 — Finding MUST have exactly one authority

Enforced. `proof.NewFinding` is the sole constructor. `pkg/assurance/authority.go` records the audit row:

Role	Package
DECIDES	<code>veriqo/pkg/proof</code> (<code>proof.NewFinding</code>)
RECORDS	<code>veriqo/pkg/casefabric</code> — <code>RecordFinding</code> writes that a finding exists and checks it belongs to an attached proof; it cannot construct one
COPIES	<code>veriqo/pkg/caseproofgraph</code> — <code>Build</code> materializes a supplied <code>Finding</code> and refuses one belonging to another proof object

Law 4 — CaseProofGraph MUST NOT become a finding authority

Was violated. Now enforced. See §2. `proof.NewFinding` is gone from the package; the finding is a parameter; a mismatched finding is refused.

Law 5 — Decision MUST have exactly one authority

Enforced, and it was already. `proof.AuthorizedFinding` has unexported fields and no constructor but `proof.Authorize`, which refuses a zero finding and refuses an authorizer who generated the proof object. `proof.Decision` is constructible only from an `AuthorizedFinding`. `caseproofgraph.AddDecision` materializes a decision node from a `Decision` the caller already holds. `casefabric` records it on the timeline. `pkg/insurance/action` derives a domain action and cannot produce a decision.

`casefabric.Outcome.Validate` reuses `proof.ProhibitedDecisionFields` rather than keeping a second list — a duplicate list is a duplicate authority with extra steps.

Law 6 — Case resolution MUST consume finalized proof

Enforced. The gate's `Decision` field is a `proof.Decision`, a type only `proof.Decide` can produce, from an `AuthorizedFinding` only `proof.Authorize` can produce, from a `Finding` only `proof.NewFinding` can produce, from a sealed object only `proof.Seal` can produce. The type chain is the enforcement; the runtime check is the second line.

Law 7 — No post-resolution epistemic mutation

Was not enforced. Now enforced. `AddEvidence`, `AddHypothesis`, `RegisterClaim`, `AttachProof` and `TestHypothesis` all refuse once the case has resolved. A case's epistemic content is frozen at the moment its resolution is computed; anything else means the record that justified the outcome is not the record that survives.

Fails if removed: `TestPostResolutionEvidenceInjection`.

Law 8 — Runtime event ordering MUST match phase contracts

Was violated. Now enforced, in two independent places. The generator verifies order and gates before emitting; `pkg/assurance/runtime_test.go` verifies the committed artefact (`TestTheRuntimeLedgerObeysTheConstitutionalSequence`, `TestTheRuntimeLedgerClosesReverseBeforeQualification`). The second check matters because it is run against the file in the repository, not against a fresh in-memory run: a stale or hand-edited artefact fails.

`casefabric.Mirror` was also corrected. It emitted the proof record at ledger index 0 — before `case.opened` — because it had no way to know where the proof belonged. It now takes the proof map and emits that record at the `proof_attached` entry, where the sequence says it goes.

Law 9 — Runtime evidence must be independently replayable

Enforced, within the honest boundary the artefact states about itself. The artefact carries:

determinism: Logical ticks, fixed identifiers, no wall-clock time and no randomness. Two runs produce identical event ids.

boundary: The evidence in this run is a fixture. It demonstrates that the chain executes and records; it does not demonstrate behaviour on real commercial data, which is the `LIVE_DATA` blocker and remains `BLOCKED_EXTERNAL`.

The integration test performs five independent re-verifications per domain (audit chain, event chain, graph, projection, replay). `TestEveryRuntimeEvidenceRefResolves` fails if the traceability matrix cites a record the run did not emit — which it did fail this round, when the indices shifted from the reordering, and the citations were corrected rather than the test relaxed.

What this does **not** establish: that an outside party has replayed it. No one outside VERIQO has. That is the L5 boundary in §5 and it has not moved.

Law 10 — All six domains must obey the same sequencing law

Enforced. The `VerifyEventOrder` and `VerifyEventGates` assertions sit inside the per-domain body of `system_integration_proof_test.go`, alongside `fwd.VerifyAgainstContract()`. Maritime, insurance, commodity, financial, legal and supply-chain each run the whole chain and each has its ledger checked against the same law. A domain that wanted its own ordering would have to defeat the same two checks six times.

4. Semantic authority audit — the deeper question

The mandate observed, correctly, that `TestNoDomainHasItsOwnChain` proves there are no duplicate **packages**, which is not the same as proving there are no duplicate **semantic authorities**. Derivation is permitted; a second decision authority is not.

`pkg/assurance/authority.go` (new, 307 lines) records eleven decisions. For each: which package **DECIDES**, which **DERIVE**, **COPY** or **RECORD**, and — the field that does the work — **why it cannot be duplicated**, stated as a structural fact rather than a promise.

Decision	Authority
Is this proof object sufficient to found a finding?	<code>pkg/proof</code> (<code>deriveSufficiency</code> , invoked only by <code>Seal</code>)
Does a finding exist for this proof object?	<code>pkg/proof</code> (<code>NewFinding</code>)

Decision	Authority
Has an authority adopted this finding?	pkg/proof (Authorize)
What operational action follows?	pkg/proof (Decide)
What is the qualification state of this claim?	pkg/qualification/state (New)
Are two sources independent?	pkg/qualification/independence (Assess)
May this recipient see this evidence?	pkg/disclosure/access (Evaluate)
What phase is this case in?	pkg/casefabric (lifecycle methods, CanTransition)
May this case resolve?	pkg/casefabric (Resolve, gated by ResolutionGate)
Does this attestation prove existence before a time?	pkg/platform/timestamp (Assess)
What level of proof has this conclusion reached?	pkg/proof (LevelOf, RaiseToExternallyAttested)

`ValidateAuthorities` refuses any entry in which a participant carries `RoleDecides` — an audit table that could quietly record a second authority would be worse than no table. `TestASecondAuthorityCannotBeDocumented` is the adversarial case.

Two rows are worth reading in full because they are the ones where the answer is not simply "one function":

- **Sufficiency.** The old worry was `pkg/proof` decides it, `pkg/casefabric` decides it, `pkg/caseproofgraph` derives it. In fact `casefabric` COPIES: `Claim.Sufficiency` is copied from the attached object in `AttachProof`, and `Claim.Proven()` reads that copy and computes nothing. The conjunctive test exists in exactly one function, and `Seal` overwrites any author-supplied value.
- **Disclosure.** `caseproofgraph.Project` calls `access.Evaluate` for evidence nodes rather than reimplementing it, and applies its own check only to structural nodes, where there is no evidence version to evaluate. (An earlier round had this wrong in a way worth recording: `Project` passed a content hash where `access.Evaluate` expects an evidence version id, so it silently withheld everything. Silently withholding looks like working security.)

5. The maturity model, as permanent law

The mandate asked that the L0–L7 model become permanent VERIQO law, and that no level be skipped. `pkg/assurance/maturity.go` (new, 307 lines) implements it.

L0 DESIGNED	the capability is specified; nothing about its behaviour follows
L1 IMPLEMENTED	the capability exists in code and builds
L2 UNIT_VERIFIED	its own tests exercise it, including the refusals
L3 INTEGRATION_VERIFIED	<- it composes with the rest of the system in an executable end-to-end proof, on fixtures
L4 REAL_DATA_VALIDATED	run on real rights-aware commercial data under a real data agreement [outside party]
L5 INDEPENDENTLY_ASSURED	a named assessor who is not VERIQO examined it and stated the procedure was correctly applied [outside party]
L6 EXTERNALLY_QUALIFIED	an accrediting body qualified it against a published standard [outside party]
L7 PRODUCTION_QUALIFIED	operated in production, under real load, with real consequences, record survives audit [outside party]

`RequiresOutsideParty()` returns true from L4 up. `InternalCeiling()` returns L3. Seventeen capabilities are recorded and **none claims above L3**:

- **L3 (10):** AI Evidence Gateway · Case Proof Graph · Case Resolution Fabric · Constitutional sequencing law · Disclosure two-dimensional model · Epistemic Qualification Fabric · Evidence Constitution (30 executable articles) · Forward-Reverse Execution Fabric · Proof Object and pipeline · Trust and Evidence Control Plane
- **L2 (3):** Redaction evidence chain · Semantic authority audit · Temporal attestation distinction
- **L1 (1):** Payment settlement
- **L0 (3):** External source acquisition (IEAP) · Operational neutrality (Article 15) · Zero-knowledge proofs

The report ends:

Highest level claimed: L3_INTEGRATION_VERIFIED. The internal ceiling is L3_INTEGRATION_VERIFIED. Nothing claims L4 or above, because every level from L4 up requires somebody who is not VERIQO, and none has been engaged.

`TestClaimingAMaturityLevelNobodyGranted` is the adversarial case: a capability declared at L4 or above without a named outside party is refused by the type, so the escalation chain the mandate named — *code exists* → *test passes* → *marketing says verified* → *investor hears certified* — is broken at the first link that leaves the repository.

6. Article 18 — the Redaction Evidence Chain, byte-level

The mandate raised Article 18 to P0 with a specific and correct demand: if VERIQO says a redacted document is irreversible, the requirement is not that a display layer hides text.

`pkg/evidence/redaction` (new, 319 lines + 234 lines of tests) implements `Verify`:

```
func Verify(original, derivative []byte,
            originalVersionID, derivativeVersionID string,
            claimedOriginalHash string,
            forbiddenTerms []string) (Result, error)
```

It searches the derivative's **bytes** for each forbidden term under twelve encodings, because a redaction that strips one representation and leaves another is not a redaction:

plain · case-folded · UTF-16LE · UTF-16BE · hex · hex (uppercase) · base64 · base64url · base32 · ascii85 · PDF hex string · NUL-interleaved

It also refuses two things that would make a pass meaningless:

- **A term that was never in the original.** Proving the absence of a word that was never there proves nothing, and would let a caller manufacture a clean report by naming terms the document never contained.
(`TestATermThatWasNeverThereProvesNothing`)
- **A derivative that is not a new version with a provenance link to a preserved original.**
(`TestTheOriginalMustBePreserved`, `TestTheDerivativeMustBeANewVersion`, `TestProvenanceIsRequired`)

`TestTheChainStatesItsOwnLimitations` requires the result to carry what it does **not** establish.

The honest status change: OPEN → INTEGRATION_GAP

Article 18 moved from OPEN to **INTEGRATION_GAP**. This is a *more precise statement of the same gap, not a smaller one*. The matrix entry says so in the source:

```
// Called is false, and that is the finding. The verifier exists
// and is tested; nothing on a live path invokes it, because no
```



```
// worker produces derivatives for it to check. The article moved
// from OPEN to INTEGRATION_GAP this round -- a more precise
// statement of the same gap, not a smaller one.
Called: false,
```

What is still missing, named plainly: the PDF, XLSX and PPTX redaction workers that would produce derivatives, and the adversarial recovery lab that would attempt reconstruction from format-specific remnants — incremental updates, object streams, revision history. A byte-absence verifier is a necessary condition for irreversibility. It is not sufficient, and this document does not claim it is.

`TestVisualOnlyRedactionPresentedAsIrreversible` and `TestRedactionThatStripsOneEncodingOnly` are the adversarial cases.

7. Audit position — what moved and what did not

	Before this round	After
OPEN	3	2
INTEGRATION_GAP	0	1
ASSURANCE_GAP	21	21
EXTERNAL_QUALIFICATION	6	6
QUALIFIED	0	0

The only movement is Article 18, OPEN → INTEGRATION_GAP, described above. Articles 9 (zero-knowledge proofs) and 15 (operational neutrality) remain OPEN and were not touched; the mandate explicitly asked that they not be closed with a fake implementation, and they have not been.

QUALIFIED remains zero. QUALIFIED requires an outside party to have examined the control. None has. No external blocker changed status this round — not the eight original production blockers, not EXTERNAL_TSA, not INDEPENDENT_ASSURANCE. Nothing in this round could change them, because every one of them requires somebody who is not VERIQO.

8. Verification

`scripts/verify.sh`, full run, exit 0:

```
== 1/6 go build ./... == pass
== 2/6 go vet ./... == pass
== 3/6 gofmt (must produce no output) == pass
== 4/6 zero-external-dependency invariant == pass
== 5/6 go test ./... -race -cover == pass
== 6/6 race-repeat on consensus-critical packages (5x) == pass
ALL CHECKS PASSED.
```

New in this round: 2,304 lines across nine new files; 1,126 insertions and 324 deletions across twenty-one modified files.

- `pkg/fref/sequence.go` — 13 tests
- `pkg/evidence/redaction` — 11 tests
- `pkg/assurance/authority.go`, `pkg/assurance/maturity.go` — authority and maturity suites

- `test/adversarial/` — 41 adversarial tests passing, of which nine are new this round:
`TestResolveFirstProveLater` · `TestReverseAsAnAfterthought` ·
`TestAnUngatedLedgerIsCaughtEvenWhenPerfectlyOrdered` · `TestPostResolutionEvidenceInjection`
· `TestGraphAsABackDoorToAFinding` · `TestVisualOnlyRedactionPresentedAsIrreversible` ·
`TestRedactionThatStripsOneEncodingOnly` · `TestClaimingAMaturityLevelNobodyGranted` ·
`TestASecondAuthorityCannotBeDocumented`

The verification script does not run `golangci-lint`, `govulncheck`, `gosec`, OPA bundle validation or SPIRE attestation; all five need network or binaries this sandbox does not have, and the script says so rather than omitting them silently.

Defects found in my own work while doing this

Recorded because a round that reports only successes is not an audit:

- A `TestAFindingFromAnotherProofObjectIsRefused` case was passing vacuously — the two fixtures were identical, so the mismatch it claimed to test could not occur. Rewritten to vary the proposition.
- The immaterial-claim test asserted the wrong semantics: a case that establishes nothing is **Closed**, not Resolved. Split into `TestImmaterialClaimIsReportedButDoesNotBlockResolution` and `TestACaseThatEstablishesNothingIsClosedNotResolved`, which say what they mean.
- A shadowed `err` in `TestReverseAsAnAfterthought` produced a nil dereference rather than the intended assertion.
- Two `gofmt` reflows silently no-op'd string-replacement edits (a `RecordReverseClosure` insertion and the `eventStep` map). Both were caught by grepping for the result rather than trusting the edit — which is the only reason they are in this list and not in the code.

9. What this round does not claim

- Reverse proof is now a gate **in the fixture-driven integration proof**. It has not been exercised on real commercial data.
- The redaction verifier is tested against twelve encodings. It has not been run against a real PDF produced by a real redaction worker, because that worker does not exist.
- The semantic authority audit is a structural argument backed by tests. It is not an independent code audit, and no independent assessor has read it.
- The five fabrics compose in one executable chain across six domains **on fixtures**. The reviewer's own status for this claim — *SUPPORTED — UNDER FIXTURE-BASED INTEGRATION PROOF* — is the correct one and is adopted here verbatim.
- Nothing in VERIQO is QUALIFIED. Nothing is above L3.

The next boundary is not another architectural round. It is L4: real rights-aware data under a real agreement, and then an assessor who is not VERIQO.